Vocabulaire :

- Les arbres et le vocabulaire associé / arbre binaire de recherche (ABR) / implémenter / spécification d'un programme / postcondition / parcours préfixe / parcours infixe / parcours suffixe / parcours en largeur / programmation itérative.

Prérequis :

- Les structures linéaires : liste, file et pile.
- Manipulation des structures de liste et des fonctions. Les listes simplement et doublement chaînée en POO.
- Avoir travaillé sur la récursivité.
- Avoir fait les séances précédentes sur les arbres et maîtrisé le vocabulaire.

Objectifs :

- Compléter un code existant sur les arbres et les pile-file.
- Savoir implémenter des méthodes en étant guider et comprendre tous les algorithmes pour parcourir un arbre : en profondeur (préfixe, infixe et suffixe) et en largeur.
- Implémenter un algorithme de recherche d'un élément dans un ABR.

Les neurones vont chauffer, accrochez-vous !

Cette séance est la quatrième concernant la structure de données **arbres**.

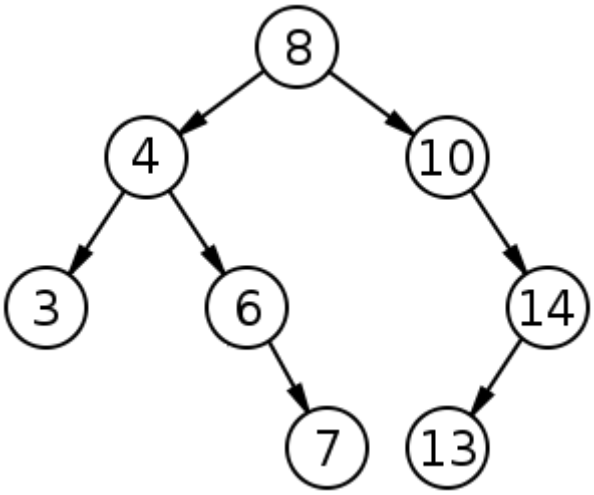
On ne s'interessera dans cette séance **aux parcours des arbres binaires de recherche**.

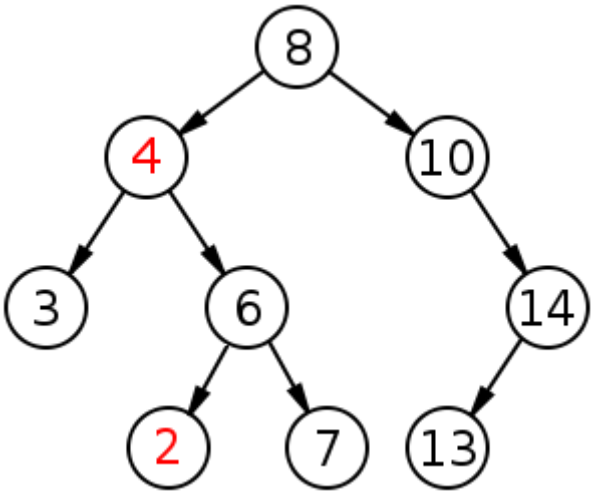
Il existe plusieurs parcours d'arbre : en profondeur par ordre préfixe, en orde infixe, en ordre postfixe, en largeur … Tu dois connaître ce vocabulaire. Nous allons apprendre à les implémenter en complétant un code comme cela se fera lors de l'examen pratique du bac.

I- UNE IMPLÉMENTATION D'UN ARBRE BINAIRE DE RECHERCHE

A_ Définition d'un arbre binaire de recherche.

- Nous avons vu ce q'était un arbre, puis un arbre binaire, voyons maintenant ce qu'est un arbre binaire de recherche que l'on abrège souvent par ABR.
- Voici une définition récursive d'un ABR :
  - Tous les valeurs des nœuds du sous-arbre gauche sont strictement inférieurs à la racine.
  - Tous les valeurs des nœuds du sous-arbre droit sont strictement supérieurs à la racine.
  - Les sous-arbre gauche et droit sont des ABR.
- L'ABR est un arbre dont les étiquettes sont classées par ordre croissant ou alphabétique.





Parmi les deux arbres binaires ci-dessus , le premier est ABR mais pas le second. Le nœud de valeur 2 devrait être en sous-arbre gauche du nœud de valeur 3.

B _ Étude du code implémentant un arbre binaire de recherche.

Sur l'ENT, tu trouves les fichiers `Arbre_parcours.py` et `PileFile.py`. Les deux fichiers sont nécessaires puisque le premier import le second pour fonctionner normalement.

- Commençons par le plus simple, `PileFile.py`. Il contient deux classes : l'une pour construire une pile et la seconde pour faire une file.

Exercice 1 :

1) Pour la classe `Pile`, complète la méthode `empiler`. Teste le programme en décommentant les lignes de tests en bas du code.
2) Pour la classe `File`, complète la méthode `traiter`. Teste le programme en décommentant les lignes de tests en bas du code.

3) Remet les lignes de tests en commentaire et copie-colle les méthodes ici.

[Copie-colle ton code ici et sauvegarde ensuite ton fichier pdf.]

- Passons au second fichier, `Arbre_parcours.py`. Il contient la classe `Arbre`.

Exercice 2 :

Voici les différentes spécifications des fonctions présentes dans la classe `Arbre`.

- Constructeur `Arbre()` •*Postconditions* : l'arbre est initialement vide
- Fonction `estVide()` : booléen •Résultat : retourne vrai si l'arbre est vide, faux sinon
- Procédure `insererElement(e)` •*Postcondition* : si e n'existe pas déjà dans l'arbre, alors un nouveau nœud contenant e est inséré en respectant la propriété des ABR, si e existe déjà dans l'arbre, alors l'arbre est inchangé.
- Fonction `hauteurArbre()` : •Résultat : la hauteur de l'arbre (longueur de sa plus longue branche), ou -1 s'il est vide.
- Procédure `afficherArbre ()` •*Postcondition*: l'arbre est affiché à l'écran

1) Cherche sur internet ce que signifie spécifier un programme et postcondition. Écris ta propre définition de ces deux termes.

[Écris ta réponse ici et sauvegarde ensuite ton fichier pdf.]

2) Tous les fonctions sont complètes à l'exception de la procédure `insererElement(e)`
   Complète-là. Il s'agit d'une fonction à comprendre et savoir réaliser pour le bac.
3) Teste ta fonction en décommentant les tests et copie-colle la fonction dans le cadre.

[Copie-colle ton code ici et sauvegarde ensuite ton fichier pdf.]

II- PARCOURS D'UN ARBRE

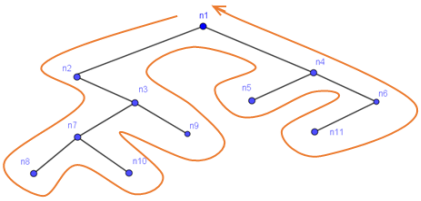
Il existe plusieurs façons de **parcourir un arbre** (= passer par tous les nœuds), nous allons en étudier quelques-unes.

Ces parcours s'appliquent sur **tous** les arbres, pas seulement les arbres binaires.

Notre implémentation fonctionnera donc, quel que soit l'arbre considéré.

A_ Les parcours en profondeur.

- Un **parcours en profondeur** est un parcours récursif de l'arbre. On partira toujours en direction du sous-arbre gauche dans nos parcours comme l'indique le schéma ci-contre.



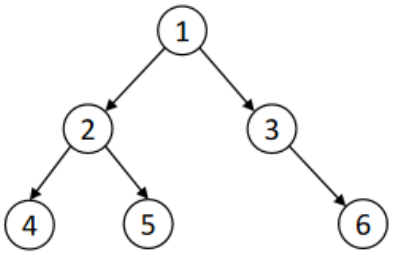
- Il y a plusieurs manières de parcourir un arbre en profondeur selon l'ordre dans lequel on affiche le nœud parcouru :
  - Dans **l'ordre préfixe** : l'affichage des nœuds se fait au moment où l'on descend et que l'on passe à gauche du nœud Une fois le dernier fils gauche traité, on remonte pour passer au fils droit le plus proche et on recommence.
    En résumé, l'ordre est : nœud, fils gauche, fils droit

- Dans **l'ordre infixe** : l'affichage se fait au moment où l'on remonte entre les nœuds gauche et droits (entre deux fils). Cela revient à descendre au nœud le plus à gauche qui n'a pas de fils et on remonte pour aller jusqu'au fils droit le plus proche et on recommence. En résumé, l'ordre est : fils gauche, nœud, fils droit

- Dans **l'ordre suffixe** : l'affichage du nœud se fait en remontant à droite du nœud. Cela revient à afficher d'abords les fils gauche et droit et puis la racine.
  En résumé, l'ordre est : fils gauche, fils droit, nœud

Cela donne sur l'arbre de droite :

- Parcours en ordre préfixe : 1 2 4 5 3 6
- Parcours en ordre infixe : 4 2 5 1 3 6
- Parcours en ordre suffixe : 4 5 2 6 3 1



Exercice 3 :

1) Voici le code d'un parcours en profondeur. Duquel s'agit-il ?

```
def affiche_parcours(self):
    if self.estVide():
        return
    if self.fg :
        self.fg.afficher_parcours()
    print(self.info, end=' ') # on traite le noeud
    if self.fd :
        self.fd.afficher_parcours()
```

[Écris ta réponse ici]

2) Copie-colle le code en modifiant son nom en `afficherParcoursPrefixe(self)`, `afficherParcoursInfixe(self)` ou `afficherParcoursSuffixe(self)` ; en fonction du type de parcours répondu à la question 1.
3) En t'inspirant du parcours précédent, implémente les deux autres parcours.
4) Teste tes fonctions en décommentant la partie test de l'exercice.
5) Copie-colle les 3 méthodes pour parcourir un arbre en profondeur :

Le parcours préfixe

Le parcours infixe

```
if self.estVide():
    return
if self.fg :
    self.fg.afficherParcoursInfixe()
print(self.info, end=' ') # on traite le noeud
if self.fd :
    self.fd.afficherParcoursInfixe()
```

Le parcours suffixe

```
def afficherParcoursSuffixe(self):
    if self.estVide():
        return
    if self.fg:
        self.fg.afficherParcoursSuffixe()   # d'abord le fils gauche
    if self.fd:
        self.fd.afficherParcoursSuffixe()   # puis le fils droit
```

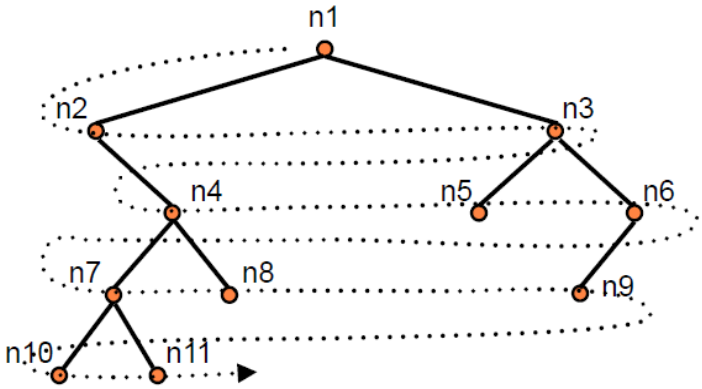
6) Les algorithmes de parcours en profondeur sont à construire le plus logiquement de manière récursive. Il est cependant possible de le faire de manière **itérative** (= avec des boucles). Voici un code de parcours itératif. Duquel s'agit-il ?

```
def afficherParcoursIteratif(self):
    if self.estVide(): return
    p = pf.Pile([])   #création d'une pile
    p.empiler(self)
    while not p.estVide():
        n = p.sommet()
        p.depiler()
        print(n.info, end=' ')
        if n.fd: p.empiler(n.fd)
        if n.fg: p.empiler(n.fg)
```

[Écris ta réponse ici]

B_ Le parcours en largeur.

- Lors du **parcours en largeur** d'un arbre, contrairement au précédent, ce parcours essaie toujours de visiter le nœud le plus proche de la racine qui n'a pas déjà été visité. En suivant ce parcours, on va d'abord visiter la racine, puis les nœuds à la profondeur 1, puis 2, etc… comme le montre le schéma ci-contre.



- Sur notre exemple, cela donne : n1, n2, n3, n4, n5, n6, n7, n8, n9, n10, n11

Exercice 4 :

1) L'algorithme se déroule ainsi :
   - Stocker le nœud racine dans une liste.
   - Afficher la racine.
   - Stocker les nœuds fils de la racine (gauche puis droit) dans la liste.
   - Afficher le nœud fils gauche de la racine puis stocker ses fils dans la liste.
   - Afficher le nœud fils droit de la racine puis stocker ses fils dans la liste.
   - … tant que la liste n'est pas vide.

   Quelle structure te semble la plus appropriée pour faire le parcours en largeur, une pile ou une file ?

   [Écris ta réponse ici] File

2) L'implémentation itérative du parcours en largeur est très similaire à l'implémentation du parcours itératif en profondeur préfixe que l'on retrouve dans la page précédente. Nous avons toutefois besoin de créer une file au lieu d'une pile. Implémente la fonction `afficherParcoursLargeur(self)` et teste la fonction.

Le parcours en largeur [Copie-colle ton code ici et sauvegarde ensuite ton fichier pdf.]

```
        return
    f = pf.File([])
    f.enfiler(self)
    while not f.estVide():
        n = f.traiter()
        print(n.info, end = '')
        if n.fg:
            f.enfiler(n.fg)
        if n.fd:
            f.enfiler(n.fd)
```

III- RECHERCHE D'UN ÉLÉMENT DANS UN ARBRE ABR

Ce travail est à réaliser seul. Il doit te permettre de te confronter à une difficulté plus importante et gérer la frustration de ne pas trouver tout de suite la solution.

Avant de te lancer, assure toi d'avoir compris tous les mots de l'énoncé.

Sujet

Implémente, toujours dans `Arbre_parcours.py`, la fonction `rechercheElement(e)`: dont le résultat est un booléen indiquant si l'élément `e` est présent dans l'arbre.

La post-condition (qui ne nécessite aucune vérification de ta part) : on recherche toujours dans un arbre ABR.

- Il existe une solution récursive et une solution itérative. Tu donneras au moins la solution **récursive**.
- Écris une phase de test pour vérifier ton code.

Copie-colle ton code dans le cadre suivant pour en garder la trace.

Version récursive	Version itérative